

3C7 Digital System Design

Lab E — Appendix Write-Up

4-bit Adder/Subtractor on the Basys3 Board

1. Integration of Code

The goal of this lab was to get a working digital design running on real hardware. Rather than jumping straight to a finished design, the approach was to build things up gradually, starting from the smallest possible component and adding layers on top.

1.1 The 1-bit Full Adder

The starting point was a simple 1-bit full adder (`adder.v`). It takes two single-bit operands and a carry-in, and produces a sum bit and a carry-out. The logic is about as basic as it gets: sum is the XOR of all three inputs, and carry-out goes high whenever at least two of the three inputs are high.

On its own it does not do much, but writing it as a separate module made the next step much cleaner. Rather than repeating the same logic four times, four copies of this module could simply be instantiated and wired together.

1.2 Scaling Up to 4 Bits

Four instances of the 1-bit adder were chained together in `adder_subtractor.v` to build a 4-bit ripple-carry adder. The carry-out of each stage feeds directly into the carry-in of the next one, which is how multi-bit addition works at the hardware level.

Subtraction was added without needing any extra hardware, by using the two's complement trick. When the sub control input is high:

- Each bit of b is inverted by XORing it with sub
- The sub signal is also used as carry-in to the first stage, which adds the +1 needed to complete the negation
- The result is that $a + (\sim b) + 1 = a - b$, giving subtraction for free

A signed overflow flag (`ovfl`) was also included. It compares the carry going into the MSB against the carry coming out of it. If they differ, the sign of the result is wrong, meaning the answer does not fit in 4 signed bits.

1.3 Getting It onto the Board

The last piece was writing the XDC constraints file to tell Vivado which physical pins on the Basys3 correspond to which module ports. The mapping used was:

- sw[3:0] for input a (rightmost 4 switches)
- sw[7:4] for input b (next 4 switches)
- sw[8] as the add/subtract control
- led[3:0] for the 4-bit result
- led[4] for carry-out, led[5] for overflow

After programming the board, flipping the switches immediately updates the LEDs, which makes it very easy to manually verify that the design is working correctly.

2. Implementation Flow

Getting Verilog onto an FPGA involves more than just pressing a button. Vivado goes through a pipeline of steps to translate the source code into something the hardware can actually run.

2.1 Synthesis

Synthesis is the first step. Vivado reads the Verilog and converts it into a technology-independent netlist of logic gates. At this stage it is not thinking about the FPGA at all, it is just figuring out what logic operations the design requires. For this adder, the XOR and AND gates fall out quite naturally from the Boolean expressions written in the source.

2.2 Implementation

Implementation is where the netlist gets physically mapped onto the Artix-7 FPGA. This happens in two stages:

- Placement: each logic gate gets assigned to a specific Look-Up Table (LUT) on the chip. A LUT is a small block that can be programmed to implement any logic function of a handful of inputs.
- Routing: once placement is done, Vivado works out how to physically connect all the LUTs together using the routing fabric on the chip.

Because this design is purely combinational (no flip-flops or clock-dependent logic), the implementation is fast and timing is not a concern.

2.3 Bitstream Generation

Once implementation is complete, Vivado generates a .bit file. This binary file encodes the full configuration of the FPGA: which LUTs are active, what function each one performs, and how they are all wired together. Loading this file onto the Basys3 via USB is the final step, and as soon as programming finishes the board is running the design.

3. Observations and Resource Consumption

3.1 Behaviour on the Board

The design worked correctly after programming. Four test cases from the simulation were verified manually on the board:

- $9 + 3 = 12$ (1001 + 0011): the sum LEDs showed 1100, which is 12 in binary. No carry-out and no overflow, as expected for a result that fits comfortably within range.
- $14 + 3 = 17$ (1110 + 0011): since 17 does not fit in 4 bits, the sum LEDs showed 0001 (the lower 4 bits of 17) with cout lighting up to indicate the carry-out. This is correct unsigned overflow behaviour.
- $10 - 3 = 7$ (1010 - 0011): the sum LEDs showed 0111, which is 7. Cout was high and ovfl was low, confirming a clean subtraction result with no borrow and no signed overflow.
- $3 - 3 = 0$ (0011 - 0011): all sum LEDs off, cout high, ovfl low. Any value minus itself correctly gives zero with no borrow and no overflow.

The board results matched the simulation outputs exactly, which confirmed that the testbench was testing the right things and the implementation was faithful to the design.

3.2 Resource Usage

The Project Summary in Vivado reported the following resource consumption after implementation on the Basys3 (xc7a35tcpg236-1):

Resource	Used	Available	Utilisation
LUTs (Logic)	9	20800	< 1%
I/O Pins	11	106	10%
Slice Registers	0	41600	0%
Block RAM	0	50	0%
DSPs	0	90	0%

The footprint is tiny, which is expected for a 4-bit combinational circuit. Nine LUTs handle all the addition, subtraction, carry, and overflow logic. There are no registers because nothing is clocked, and no DSPs or block RAM because this is purely simple gate logic. The eleven I/O pins account for the four switches of a, four of b, the sub switch, the four sum LEDs, carry-out, and overflow, plus the clock pin that Vivado requires even when it is not actually used by the design.